

Assignment # 222508-CS57

Objectives:

1. Containerization of your applications
 - a. Dockerization
 2. Create any application that uses a database & (optionally) other components such as a Message broker, Cache etc.
 3. You need to port your project on a different machine, maybe, running a diff OS / Arch.
 4. Avoid recreating your environment (such as installation of those things) as best you can
 5. The PO wishes to have all these such streamlined that they could be run anywhere on-demand, plus he should be able to spin up as many instances of your application as he desires.
-

What Is Docker?

Docker is a **containerization platform** that lets you package applications and their dependencies into lightweight, portable containers. These containers run consistently across environments—whether it's your laptop, a server, or the cloud.

Why Use Docker?

-  Consistent environments (no “works on my machine” issues)
 -  Lightweight and fast
 -  Easy to scale and deploy
 -  Great for microservices and CI/CD pipelines [**Don't bother about it now**]
-

Docker Installation Guide (Step-by-Step)

Step	Action	Description
1	Go to Docker website	Visit docker.com
2	Download Docker Desktop	Choose Windows, Mac, or Linux version

- | | | |
|----------|-----------------------------|--|
| 3 | Run installer | Follow on-screen instructions |
| 4 | Enable WSL 2 (Windows only) | Docker Desktop may prompt to enable WSL 2 backend |
| 5 | Launch Docker Desktop | Start the app and wait for it to initialize |
| 6 | Verify installation | Run <code>docker --version</code> in terminal |
| 7 | Test Docker | Run <code>docker run hello-world</code> to confirm setup |
-

MySQL Setup in Docker

Let's spin up a MySQL container using Docker—no need to install MySQL locally!

Pull MySQL Image

```
docker pull mysql:8.0
```

Run MySQL Container

```
docker run --name mysql-db \\  
-e MYSQL_ROOT_PASSWORD=rootpass \\  
-e MYSQL_DATABASE=mydb \\  
-e MYSQL_USER=myuser \\  
-e MYSQL_PASSWORD=mypass \\  
-p 3306:3306 \\  
-d mysql:8.0
```

Breakdown of Flags

Flag	Purpose
--name	Container name
-e	Set environment variables (DB name, user, password)
-p	Map port 3306 to host
-d	Run in detached mode

mysql:8.0 Image version

Verify It's Running

`docker ps`

Connect to MySQL

Use any MySQL client (e.g., MySQL Workbench, DBeaver) or CLI:

```
mysql -h 127.0.0.1 -P 3306 -u myuser -p
```

Basic Docker Commands

Command	Example	Explanation
<code>docker --version</code>	<code>docker --version</code>	Check installed Docker version
<code>docker pull</code>	<code>docker pull ubuntu</code>	Download image from Docker Hub
<code>docker images</code>	<code>docker images</code>	List downloaded images
<code>docker run</code>	<code>docker run ubuntu</code>	Run a container from an image
<code>docker ps</code>	<code>docker ps</code>	List running containers
<code>docker ps -a</code>	<code>docker ps -a</code>	List all containers (including stopped)
<code>docker stop</code>	<code>docker stop mycontainer</code>	Stop a running container
<code>docker start</code>	<code>docker start mycontainer</code>	Start a stopped container
<code>docker rm</code>	<code>docker rm mycontainer</code>	Remove a container
<code>docker rmi</code>	<code>docker rmi ubuntu</code>	Remove an image
<code>docker exec</code>	<code>docker exec -it mycontainer bash</code>	Run command inside running container
<code>docker logs</code>	<code>docker logs mycontainer</code>	View container logs
<code>docker build</code>	<code>docker build -t myimage .</code>	Build image from Dockerfile

docker-
compose up

docker-compose up Start services defined in [docker-compose.yml](#)

Stop and remove services from [docker-compose.yml](#)

mysql -h 127.0.0.1 -P 3306 -u myuser -p

Java Application on Tomcat in Docker

Let's containerize a Java web application that runs on Apache Tomcat.

Create Dockerfile

FROM tomcat:9.0

Remove default web apps

RUN rm -rf /usr/local/tomcat/webapps/*

docker-
compose down

docker-compose
down

Copy your WAR file into the container

COPY your-app.war

/usr/local/tomcat/webapps/ROOT.war

EXPOSE 8080

Build Docker Image

docker build -t java-tomcat-app .

Run Container

docker run --name tomcat-app -p 8080:8080 -d
java-tomcat-app

Verify It's Running

docker ps

Access your app at <http://localhost:8080>